The derivatives of `SolutionSpaceFeature` included features that measured whether the appropriate spaces were placed into the solution, whether the spaces had the appropriate area and aspect ratio, whether elevators stacked properly, etc.

The neat thing about this is that the loop that traverses the data structure is located in one place. All the scoring features inherit it rather than reimplement it.

Some of the features had to measure characteristics of the portals attached to a space. So we reproduced the pattern and created the class `PortalFeature` derived from `SolutionSpaceFeature`. The implementation of `NewSolutionSpace` within `PortalFeature` looped through all the portals in the `SolutionSpace` argument and called the pure virtual function `NewPortal(const Portal&)`. (See Figure 30-5.)

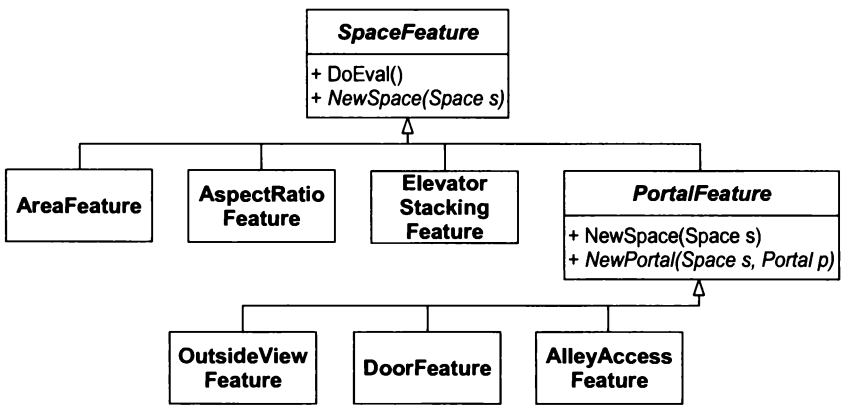


Figure 30-5 TEMPLATE METHOD structure of the scoring features

This structure allowed us to create dozens of different scoring features, each one of which walked the floor-plan data structure, without knowing what the floor-plan data structure looked like. If the details of the floor-plan data structure changed (e.g., we decided to use STL instead of our own iterators), we'd have to change two classes rather than several dozen.

Why did we choose TEMPLATE METHOD over STRATEGY?[5] Consider how much looser the coupling would have been had we used STRATEGY! (See Figure 30-6.)

Using the TEMPLATE METHOD structure, if we had to make a change to the algorithms that walked the data structure, we would have had to change `SpaceFeature` and `PortalFeature`. In all likelihood, this would have forced us to recompile *all* the features. However, using the STRATEGY pattern, the change would have been restricted to the two `Driver` classes. There is virtually no chance that the features would need to be recompiled.

So why did we choose the TEMPLATE METHOD? Because it was simpler. Because the data structure was not something that was going to change frequently. And because recompiling all the features cost just a few minutes.

So, even though the use of inheritance in the TEMPLATE METHOD pattern resulted in a more tightly coupled design, and even though the STRATEGY pattern conforms to the DIP better than the TEMPLATE METHOD pattern does, in the end *it wasn't worth the two extra classes* to implement the STRATEGY.

The Common Requirements of the Delivery Applications

The delivery programs had quite a bit of overlap. For example, the structure of the screen was the same in all vignettes. At the left of the screen was a window that contained nothing but a column of buttons. This window was called the "Command Window." The buttons in the Command Window acted as the controls for the application. They were labeled with terms such as, "Place Item," "Erase," "Move/Adjust," and "Zoom," and "Done." Clicking on these buttons drove the application through the desired behaviors.

5. Clearly, we didn't think of it in those terms. The names of the patterns hadn't been invented at the time we made this decision.

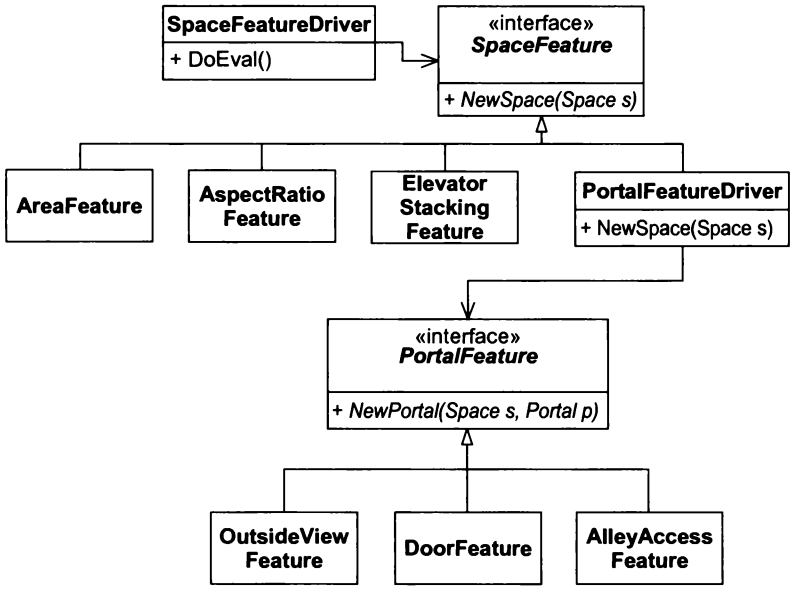


Figure 30-6 Floor-plan scoring structure with STRATEGY

To the right of the Command Window was the Task Window. This was a large scrollable and zoomable area into which the user could draw his solution. Commands initiated in the Command Window were typically used to modify the contents of the Task Window. Indeed, most of the commands that were initiated in the command window required significant interaction within the Task Window.

For example, in order to place a room on a floor plan, the user would click on the Place Item button in the Command Window. A menu of possible rooms would pop up. The user would select the kind of room he wished to place into the floor plan. Then the user would move the mouse into the Task Window and click at the location where he wanted the room to be placed. Depending upon the vignette, this might anchor the upper left corner of the room where the user clicked. The lower left corner of a stretchable room would then follow the motions of the mouse in the Task Window until the user clicked for the second time, anchoring the lower left corner at that position.

These actions were similar, though not identical, in each vignette. Some vignettes did not deal with rooms, but rather dealt with contour lines, property lines, or roofs. Though there were differences, the overall paradigm of operating within the vignettes was quite similar.

This similarity meant that we had a significant opportunity for reuse. We ought to be able to create an object-oriented framework that captured the bulk of the similarities and that allowed the differences to be expressed conveniently. In this we succeeded.

The Design of the Delivery Framework

The ETS framework finally grew to be nearly 75,000 lines of code. Clearly we cannot show all the details of that framework here. Thus, we have chosen two of the most illustrative elements of the framework to explore: the event model and the taskmaster architecture.

The Event Model. Every action taken by the user caused events to be generated. If the user clicked upon a button, an event named after that button was generated. If the user selected a menu item, an event named for that menu item was generated. Marshalling those events was a significant problem for the framework.

The reason for the problem was the fact that a very large portion of all the events could be handled by the framework, and yet each individual vignette might need to override the way the framework handled a particular

event. Thus, we needed to find a way to allow the vignettes the power to override the processing of events if needed.

The problem was complicated by the fact that the list of events was not closed. Each vignette could select its own particular set of buttons in the Command Window and its own particular set of menu items. Thus, the framework needed to marshal the events that were common to all vignettes, while allowing each vignette to override the default processing; and it needed to allow for the vignette to marshal its own vignette-specific events. This was not an easy task to accomplish.

As an example, consider Figure 30-7. This diagram[6] shows a small portion of the finite state machine that would be used to marshal the events that occurred in the Command Window of a vignette. Each vignette had its own special version of this finite state machine.

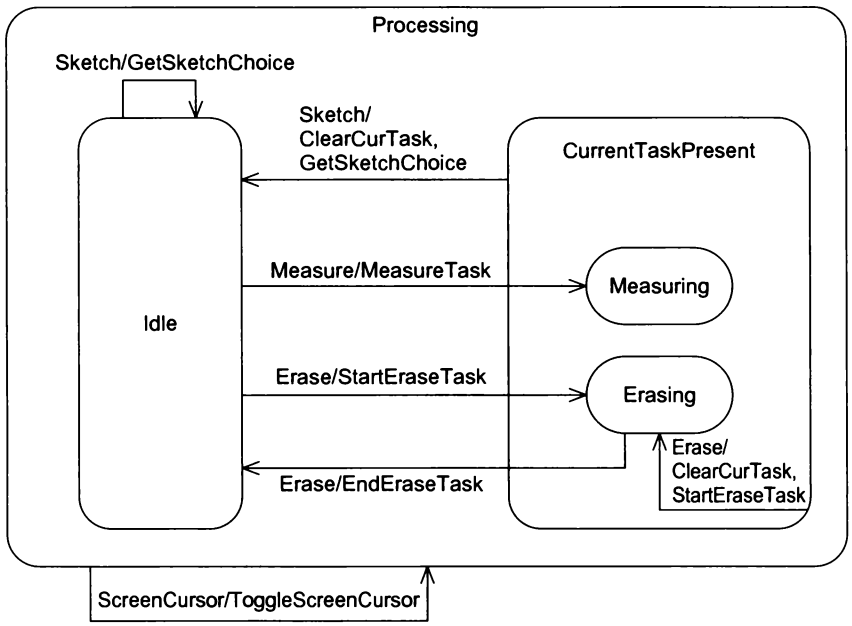


Figure 30-7 Command Window Event Processor

Figure 30-7 shows how three different kinds of events behaved. Let us first consider the simplest case, the `ScreenCursor` event. This event was generated when the user clicked on the Change Cursor button. Each time the user clicked this button, the cursor in the Task Window would toggle between an arrow and a full-screen cross hair. Thus, although the cursor changed state, no change of state occurred in the event processor.

When a user wants to delete an object he has drawn, he clicks on the Erase button. He then clicks on the item or items in the Task Window that he wants to delete. Finally, he clicks again on the Erase button to commit the deletion. The state machine in Figure 30-7 shows how the Command Window event processor deals with this. The first `Erase` event causes a transition from the `Idle` state to the `Erasing` state and starts up an `Erase Task`. We'll be talking more about `Tasks` in the next section. For now, it is sufficient that you know that the `Erase Task` will deal with any events that take place in the Task Window.

Notice that even in the `Erasing` state, the `ScreenCursor` event will still function properly and will not interfere with the erase operation. Notice also that there are two ways out of the `Erasing` state. If another `Erase` event occurs, then the `Erase Task` is ended, committing the erasure, and the state machine transitions back to the idle state. This is the normal way to end the erase operation.

6. The notation for state diagrams like this is fully described in Appendix B on page 489.

The other way to end the erase operation is to click on some of the other buttons in the Command Window. If, while erasure is in progress, you click on a Command Window button that starts up a different task (e.g., the Sketch button) then the `Erase Task` is aborted, and the deletion is cancelled.

Figure 30-7 shows how this cancellation works with the `Sketch` event, but there are many other events not shown that work the same way. If the user hits the Sketch button, either when the system is in the `Erasing` or `Idle` state, the system will transition to the `Idle` state and the `GetSketchChoices` function will be called. This function puts up the sketch menu, which contains a list of operations that the user can perform. One of those operations is measure.

When the user selects the measure item from the sketch menu, the `Measure` event occurs. This starts up the `Measure Task`. When measuring, the user may click on two points in the Task Window. The two points will be marked with tiny little cross hairs, and the distance between them will be reported in a small message window at the bottom of the screen. The user may then click on two more points, and two more, and two more, etc. There is no normal exit from the `Measure Task`. Instead, the user must click on a button that could start up another task, like Erase or Sketch.

The Event Model Design. Figure 30-8 shows the static model of the classes that implement the Command Window event processor. The hierarchy on the right represents the `CommandWindow`, while the hierarchy on the left represents the finite-state machine that translates events into actions.

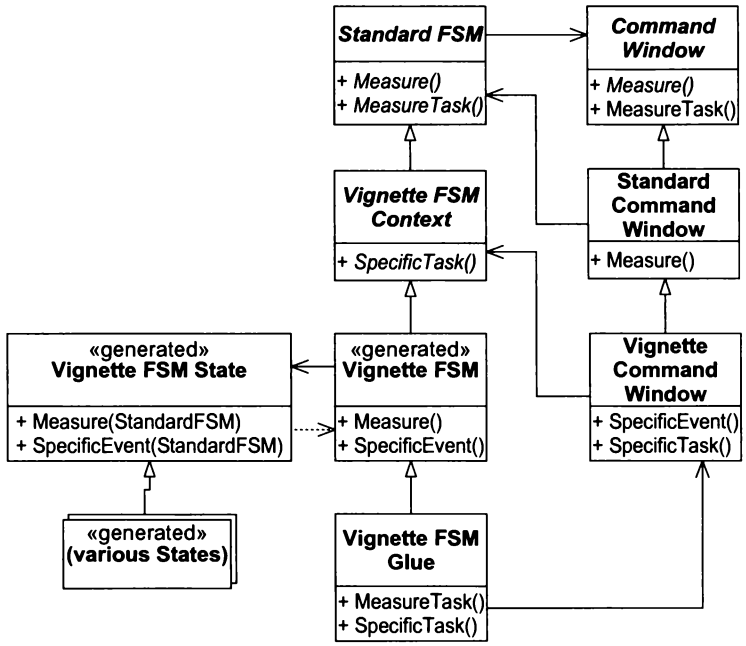


Figure 30-8 Command Window Event Processor Static Model

`CommandWindow`, `StandardCommandWindow`, and `StandardFSM` are framework classes. The rest are specific to the vignette. `CommandWindow` provides the implementations for standard actions, such as `MeasureTask` and `EraseTask`.

Events are received by the `VignetteCommandWindow`. They are passed to the finite-state machine, which translates them into actions. The actions are then passed back to the `CommandWindow` hierarchy, which implements them.

The `CommandWindow` class provides implementations for standard actions such as `MeasureTask` and `EraseTask`. A "standard action" is an action that is common to all vignettes. `StandardCommandWindow`

provides for the marshalling of standard events to the finite-state machine. `VignetteCommandWindow` is specific to the vignette and provides for both the implementation of specific actions and the marshalling of specific events. It also allows the standard implementations and marshalling to be overridden.

Thus, the framework provides default implementations and marshalling for the all the common tasks. But any one of those implementations or marshallings can be overridden by the vignette.

Tracing a Standard Event. Figure 30-9 shows how a standard event gets marshalled to the finite state machine and translated to a standard action. Message 1 is the `Measure` event. It comes from the GUI and is passed to the `VignetteCommandWindow`. The default marshalling for this event is provided by the `StandardCommandWindow`, so it forwards the event to the `StandardFSM` in message 1.1.

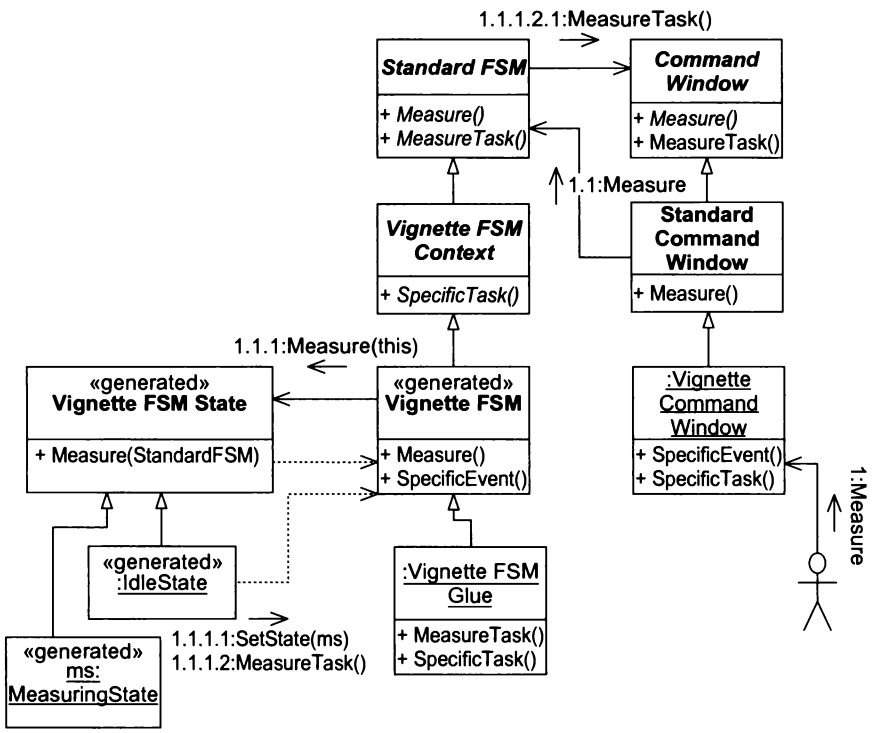


Figure 30-9 Processing the Measure Event

`StandardFSM` is a framework class that provides an interface for all the incoming standard events and all the outgoing standard actions. None of these functions are implemented at this level. `VignetteFSMContext` adds interfaces, but no implementations, for vignette-specific events and actions.

The real work of translating an event to an action takes place in the `VignetteFSM` and `VignetteFSMState` classes. `VignetteFSM` contains the implementations for all the event functions. So, the `1.1:Measure` message gets deployed down to this level. `VignetteFSM` responds by sending `1.1.1:Measure(this)` to the `VignetteFSMState` object.

`VignetteFSMState` is an abstract class. There are derivatives of this class for each state of the finite-state machine. In Figure 30-9, we presume that the current state of the FSM is `Idle`. (See Figure 30-7.) Thus the `1.1.1:Measure(this)` message gets deployed to the `IdleState` object. This object responds by sending two messages back to `VignetteFSM`. The first is `1.1.1.1:SetState(ms)`, which changes the state of the FSM to the `Measuring` state. The second message is `1.1.1.2:MeasureTask()`, which is the action required by the `Measure` event in the `Idle` state.

The `MeasureTask` message is finally implemented in the `VignetteFSMGlue` class, which recognizes the action as a standard action declared in `CommandWindow`, and therefore routes it there in message `1.1.1.2.1:MeasureTask`, thus completing the circuit.

The mechanism employed for converting events to actions is the STATE pattern. We make prodigious use of this pattern in many areas of the framework, as the following sections will show. The «generated» stereotype that appears in the STATE pattern classes indicates that these classes are automatically generated by SMC.

Tracing a Vignette Specific Event. Figure 30-10 shows what happens when a vignette-specific event occurs. Once again, message `1:SpecificEvent` is caught by the `VignetteCommandWindow`. However, since the marshalling for specific events are implemented at this level, it is the `VignetteCommandWindow` that sends message `1.1:SpecificEvent`. Moreover, it sends it to the `VignetteFSMContext` class where the `SpecificEvent` method was first declared.

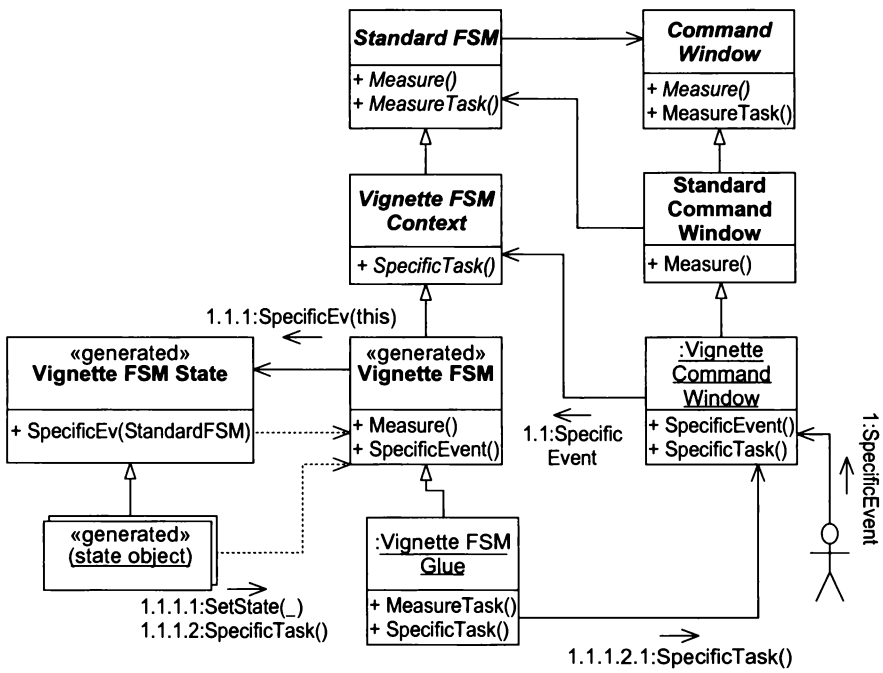


Figure 30-10 Processing a specific event

Once again, the event is deployed to `VignetteFSM`, which, in message `1.1.1:SpecificEv`, negotiates with the current state object to generate the corresponding action. As before, the state object replies with two messages, `1.1.1.1:SetState` and `1.1.1.2:SpecificTask`.

Again, the message is deployed down to `VignetteFSMGlue`. However, this time, it is recognized as a vignette specific action and is therefore routed directly to the `VignetteCommandWindow` where specific actions are implemented.

Generating and Reusing the Command Window State Machine. At this point, you might be wondering why it requires so many classes to marshal the events and actions in this fashion. Consider, however, that, although there are many classes, there are very few objects. Indeed, the instantiated objects are simply `VignetteCommandWindow`, `VignetteFSMGlue`, and the various state objects, which are trivial and automatically generated.

Though the flow of messages seems complex, it is really quite simple. The window detects an event, passes it to the FSM for translation to an action, and accepts the action back from the FSM. The rest of the complexity has to do with separating standard actions known to the framework from specific actions known to the vignette.

Another factor impinging on our decision to partition the classes as we did was our use of SMC to automatically generate the finite-state machine classes. Consider the following description, and refer back to Figure 30-7:

```
Idle
{
  Measure Measuring MeasureTask
  Erase   Erasing   StartEraseTask
  Sketch  Idle      GetSketchChoice
}
```

Notice that this simple text describes all of the transitions that occur while the state machine is in the `Idle` state. The three lines in the braces identify the event that triggers the transition, the target state of the transition, and the action performed by the transition.

SMC[7] accepts text in this form and generates the classes denoted with «generated». The code generated by SMC requires neither editing nor any inspection.

Using SMC to generate the state machine makes the creation of this part of the event processor for a vignette quite simple. The developer must write the `VignetteCommandWindow` with its corresponding implementations for specific events and actions. The developer must also write the `VignetteFSMContext`, which simply declares interfaces for the specific events and actions. And then the developer needs to write the `VignetteFSMGlue` class, which simply dispatches actions back to `VignetteCommandWindow`. None of these tasks are particularly challenging.

There is one other thing the developer must do. He must write the description of the finite-state machine for SMC. This state machine is actually rather complicated. The diagram in Figure 30-7 doesn't do it justice at all. A real vignette must deal with many dozens of different events, each of which can have remarkably different behaviors.

Fortunately, most of the vignettes behave in roughly the same way. So we were able to use a standard state-machine description as a model and make relatively minor modifications to it for each vignette. Thus, each vignette had its own FSM description.

This approach is somewhat unsatisfying since the FSM description files were very similar. Indeed, there were several occasions when we were forced to change the generic state machine, which meant that we had to make identical or nearly identical changes in each of the FSM description files. This is both tedious and error prone.

We might have invented yet another scheme for separating the generic portion of the FSM description from the specific portion, but in the end we didn't feel it was worth the effort. This is a decision we have chided ourselves for more than once.

The Taskmaster Architecture

We have seen how events get converted into actions and how that conversion is dependent on a relatively complex finite-state machine. Now we are going to look at how the actions themselves are processed. It should come as no surprise that the core of each action is also driven by a finite state machine.

Let's consider the `MeasureTask` that we discussed in the previous section. The user invokes this task when he wants to measure the distance between two points. Once invoked, the user clicks on a point in the `TaskWindow`. A small cross hair will appear at that point. As the user then moves the mouse around, a stretchable line is drawn

7. SMC, the State Machine Compiler, is freeware available from `http://www.objectmentor.com`.

from the click point to the current mouse position. Moreover, the current length of that line is displayed in a separate message window. When the user clicks for the second time, another cross hair is drawn, the stretchable line disappears, and the final distance between the two points is displayed in the message box. If the user then clicks again, the process starts over.

Once the event processor has selected the `MeasureTask`, as shown in Figure 30-9, the `CommandWindow` creates the actual `MeasureTask` object, which then runs the finite-state machine shown in Figure 30-11.

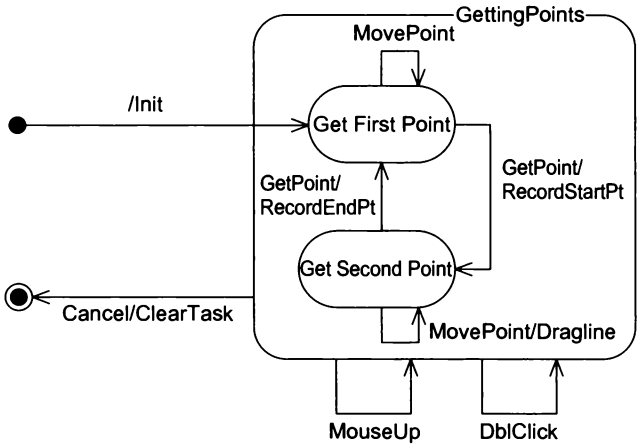


Figure 30-11 Finite-State Machine for the Measure Task.

The `MeasureTask` begins its life by invoking its `init` function and then occupying the `GetFirstPoint` state. GUI events that occur in the `TaskWindow` are routed to the task that is currently executing. Thus, when the user moves the mouse in the `TaskWindow`, the current task receives a `MovePoint` message. Notice that, in the `GetFirstPoint` state, this does nothing (as expected).

When the user finally clicks in the `TaskWindow`, a `GotPoint` event occurs.This causes a transition to the `GetSecondPoint` state and invokes the `RecordStartPt` action. This action will draw the first cross hair and will also remember the location that the mouse was clicked as the starting point.

In the `GetSecondPoint` state, a `MovePoint` event causes the `Dragline` action to be invoked. This action sets `XOR` mode[8] and draws a line from the remembered starting point to the current mouse location. It also computes the distance between these two points and displays it in the message window.

`MovePoint` events occur frequently whenever the mouse is being moved over the `TaskWindow`, so the motion of the line, and the length displayed in the message window, will appear to continuously update as long as the mouse is in motion.

When the user clicks for the second time, we transition back to the `GetFirstPoint` state and invoke the `RecordEndPt` action. This action turns off `XOR` mode, erases the line between the first point and the current mouse position, draws a cross hair at the click point, and displays the distance between the starting point and the click point in the message window.

This sequence of events repeats as long as the user desires. It only terminates when the task is cancelled by the `CommandWindow`, probably in response to the user clicking on a command button.

Figure 30-12 shows a somewhat more complex task—that of drawing a "two-point" box. A two-point box is a rectangle that is drawn on the screen with two clicks. The first click anchors one corner of the box. A stretchable box then follows the mouse. When the user clicks for the second time, the box is made permanent.

8. XOR mode is a mode in which the GUI can be placed. It vastly simplifies the problem of dragging stretchable lines or shapes over existing shapes on the screen. If you don't understand this, don't worry about it.

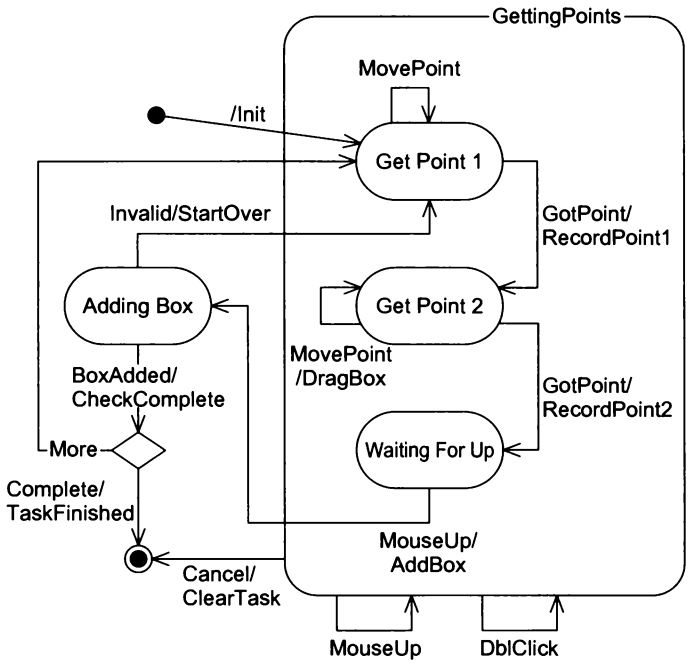


Figure 30-12 Two-Point Box

As before, the task begins in the `GetPoint1` state, after invoking `Init`. In this state, the motion of the mouse is ignored. When the mouse is clicked, we transition to `GetPoint2` and invoke the `RecordPoint1` action. This action records the click point as the starting point.

In the `GetPoint2` state, mouse motion causes the `DragBox` action to be invoked. This function sets `XOR` mode and draws the stretchable box from the starting point to the current mouse position.

When the mouse goes down for the second time, we transition to the `WaitingForUp` state and invoke `RecordPoint2`. This function simply records the final point for the box. It does not cancel `XOR` mode, erase the stretchable box, or draw the real box, because we aren't sure if the box is valid yet.

At this point, the mouse is still down, and the user is about to lift his finger off the mouse button. We need to wait for this to occur, otherwise some other task might get the mouse-up event and confuse things. While we are waiting, we ignore any motion of the mouse, leaving the box anchored at the last click point.

Once the mouse comes up, we transition to the `AddingBox` state and invoke the `AddBox` function. This function checks to see if the box is valid. There are many reasons why a box might be invalid. It might be degenerate (i.e., its first and final points are the same), or it might conflict with something else on the drawing. Each vignette has the right to reject something that the user has attempted to draw.

If the box is found to be invalid, an `Invalid` event is generated, and the state machine transitions back to `GetPoint1` while invoking the `StartOver` function. If, however, the box was found to be valid, then the `BoxAdded` event is generated. This causes the `CheckComplete` function to be called. This is another vignette-specific function. It determines whether the user should be allowed to continue drawing more boxes or whether the task should be completed.

There are literally dozens of such tasks in the framework. Each is represented by a derivative of the `Task` class. (See Figure 30-13.) Each task has a finite-state machine within it, and they are substantially more complex than we have been able to show here. Again, each of those state machines was generated by SMC.

Figure 30-13 shows the Taskmaster architecture. This architecture connects the `CommandWindow` to the `TaskWindow`, and creates and manages the tasks that the user has selected.

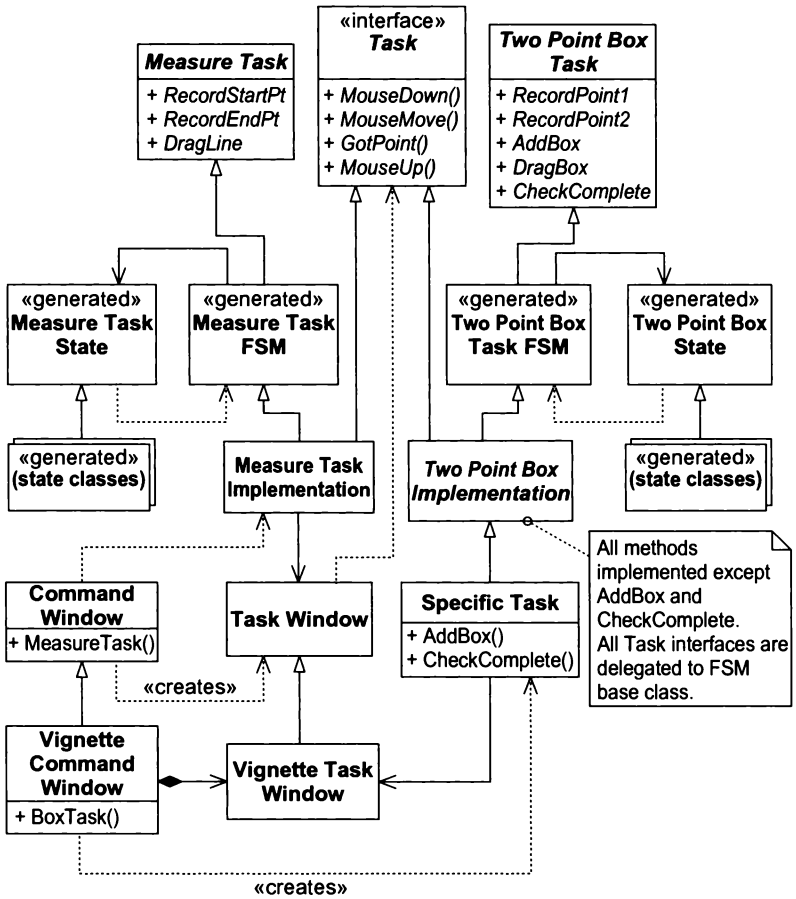


Figure 30-13 Taskmaster Architecture

The diagram shows the two tasks whose FSMs were depicted in Figures 30-11 and 30-12. Notice the use of the STATE pattern and the classes generated in each of the tasks. All of these classes, down to `MeasureTaskImplementation` and `TwoPointBoxImplementation` are part of the framework. Indeed, the only classes that the developer must write are `VignetteTaskWindow` and the specific derivatives of the task classes.

The classes `MeasureTaskImplementation` and `TwoPointBoxImplementation` represent the many different tasks that are contained in the framework. But note that these classes are abstract. There are a few functions, such as `AddBox` and `CheckComplete`, that are not implemented. Each vignette must implement these functions as needed for their own concerns.

Thus, the tasks contained by the framework govern the bulk of the interactions in all the vignettes. Whenever a developer needs to draw a box, or an object related to a box, that developer can derive a new task from `TwoPointBoxImplementation`. Or whenever he needs to simply place some object on the screen with a single click, he can override `SinglePointPlacementTask`. Or if he needs to draw something based upon a polyline, he can override `PolylineTask`. And these tasks manage the interactions, perform any needed dragging, and give the engineer the hooks needed to validate and create the objects he needs.

Conclusion

Of course there is much more that we could have talked about in this chapter. We could have discussed the elements of the framework that dealt with computational geometry or the bits that dealt with storing and reading the

solutions to and from files. We could have discussed the structure of the parameter files that allowed each vignette application to drive many different variations of the same vignette. Unfortunately, neither space nor time permits.

However, we think we covered the aspects of the framework that were most instructional. The strategies we employed in this framework can be employed by others to make their own reusable frameworks.

Bibliography

1. Booch, Grady. *Object-Oriented Design with Applications*. Redwood City, CA: Benjamin Cummings, 1991.
2. Booch, Grady. *Object Solutions*. Menlo Park, CA: Addison–Wesley, 1996.
3. Gamma, et al. *Design Patterns*. Reading, MA: Addison–Wesley, 1995.

APPENDIX A

UML Notation I: The CGI Example

The analysis and design of software comprise a process that cries out for some kind of notation. There have been many attempts at creating such a notation. Flowcharts, data-flow diagrams, entity-relationship diagrams, etc.

The advent of object-oriented programming saw an explosion in notation. There were literally dozens of competing notations for the representation of object-oriented analyses and designs.

The most popular of these notations were the following:

- Booch 94.[1]
- OMT (Object Modeling Technique) as described by Rumbaugh, et al.[2]
- RDD (Responsibility Driven Design) as described by Wirfs-Brock, et al.[3]
- Coad/Yourdon as described by Peter Coad and Ed Yourdon[4]

Of these, Booch 94 and OMT were by far the most important. Booch 94 was hailed as a strong design notation, whereas OMT was considered to be stronger as an analysis notation.

This dichotomy is interesting. In the late 1980s and early 1990s, it was considered one of the advantages of object orientation that analysis and design could be represented by the same notation. Possibly, this was a reaction to the strong separation between the notations for structured analysis and structured design. Crossing the chasm from structured analysis to structured design was well known to be difficult.

When object-oriented notations first exploded upon the scene, it was felt that the same notation would serve both analysis and design. Yet as the decade progressed, analysts and designers began to migrate to their favorite notations. Analysts tended to favor OMT, and designers tended to favor Booch 94. Thus, it appeared that one notation would not really suffice. A notation tuned for analysis was not appropriate for design, and vice versa.

The UML is a single notation, but it has broad applicability. Parts of the notation are usable for analysis; other parts are usable for design. Thus, both analysts and designers can use the UML.

In this chapter, we will be presenting the UML notation from both views. First we will describe an analysis, and then we will go on to describe a design. This description will take place in the form of a miniature case study.

1. [BOOCH94].
2. [RUMBAUGH91].
3. [WIRFS90].
4. [COAD91A].

Please note that this ordering of analysis first, and then design, is artificial, and is not intended as a recommendation. Indeed, none of the other case studies in this book makes the distinction between the two. I have presented it this way here simply to illustrate how UML can be used at different levels of abstraction. In a real project all levels of abstraction are produced concurrently—not in sequence.

Course Enrollment System: Problem Description

Suppose that we are working for a company that offers professional training courses in object-oriented analysis and design. The company needs a system that keeps track of the courses being taught and the students that are enrolled. See the "Course Enrollment System" sidebar.

Course Enrollment System

Users must be able to view a menu of the available course offerings and select the course in which they wish to enroll. Once selected, a form should pop up that allows the user to enter the following information:

- Name
- Phone number
- Fax number
- e-mail address

There should be a way for the user to select the manner in which he wants to pay for the course. Those methods may be one of the following:

- Check
- Purchase order
- Credit card

If the user wishes to pay by check, then the form should prompt him for the check number.

If the user wishes to pay by credit card, then the form should prompt him for the credit card number, expiration date, and the name as it appears on the card.

If the user wishes to pay by purchase order, then the form should prompt him for the purchase order number (PO#), the name of the company, and the name and phone number of someone in the accounts payable department.

Once all this information has been filled out, the user will click on a 'submit' button. Another screen will pop up, which summarizes all the information that the user entered. It will instruct the user to print the screen, sign the printed copy, and fax it to the number of the enrollment center.

It should also e-mail an enrollment summary to our enrollments clerk and to the user.

The system will know the maximum number of students for each class, and will automatically mark the class offering as "Sold Out" once that limit has been hit.

The enrollments clerk will be able to e-mail messages to all students enrolled in a given course by bringing up a special form and selecting the course. This form will allow the clerk to type a message and then hit a button that will send it to all students currently enrolled in the selected course.

(continues on next page)

The enrollments clerk will also be able to bring up a form that shows the status of all students for classes that have already been taught. The status will indicate whether or not the student was in attendance and whether or not the student's payment has been received. This form can be brought up on a course-by-course basis. Or the enrollments clerk can ask to see a list of all students who have outstanding balances.

Identifying Actors and Use Cases. One of the tasks of requirements analysis is to identify the actors and use cases. It should be noted that in a real system, these are not necessarily the first tasks that are appropriate. But for the purposes of this chapter, they are where I choose to begin. In reality, the place you begin is less important than the act of beginning.

Actors

Actors are entities that interact with the system, but are outside the system. Often they are roles played by the users of the system. Sometimes, however, they can be other systems. In this example, all the actors correspond to human users.

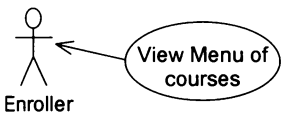
Enroller. This actor enrolls a student in a course. It interacts with the system to select the appropriate course and enter the information about the student and the method of payment.

Enrollment Clerk. This actor receives e-mail notification of each enrollment. It also sends e-mail notices to students and receives reports about enrollments and payments

Student. This actor receives e-mail confirmation of enrollment and email notifications from the Enrollment Clerk. The Student attends the courses it is enrolled in.

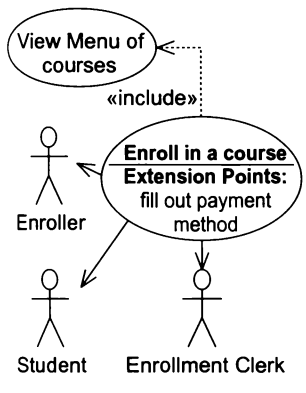
Use Cases

Having determined the actors, we specify the interactions of these actors with the system. These specifications are called "use cases." A use case describes the interaction between an actor and the system from the actor's point of view. None of the inner workings of the system are discussed, nor is the user interface described in any detail.



Use Case #1: View Menu of Courses. The Enroller requests a list of courses that are currently available in the course catalog. The system displays the list of courses. Included in that list are the name, time, place, and cost of the course. The list will also show the number of students allowed in the course and whether or not the course is currently sold out.

Use Case Notation. The above diagram shows an actor and a use case within a use-case diagram. The actor is the little stick figure, and the use-case is the ellipse. The two are bound together by an association that shows the direction of the data flow.



Use Case #2: Enroll in a course. The Enroller first views the menu of courses (Use Case #1). The Enroller selects a course for enrollment from the menu. The system prompts the Enroller for the name, phone number, fax number, and email address of the student. The system also prompts the Enroller for the preferred payment method.

Extension point: fill out payment method

The Enroller submits the enrollment form. The Student and the Enrollment Clerk are sent email confirming the enrollment. The Enroller is shown a confirmation of the enrollment and is asked to print that confirmation, sign it, and fax it to a specific fax number.

Extending and Using Use Cases. Use Case #2 has an extension point. This means that other use cases will be extending this use case. The extending use cases are presented below as #2.1, #2.2, and #2.3. Their descriptions are inserted into the previous use case at the extension point. They describe the optional data that need to be entered depending on the selected payment method.

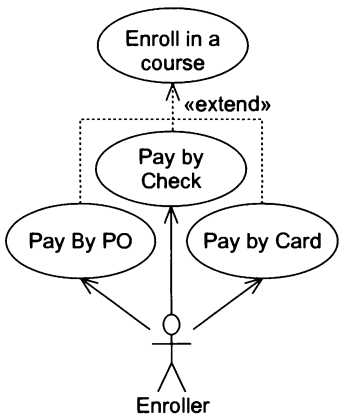
Use Case #2 also has a «include» relationship with Use Case #1, "View Menu of Courses." This means that the description of Use Case #1 is inserted into the appropriate location of Use Case #2.

Note the difference between extending and including. When one use case includes another, the including use case references the included use case. However, when one use case extends another, neither use case refers to the other. Instead, an extending use case is selected based upon context and is inserted into the extended use case at the appropriate point.

We use the «include» relationship when we want to make the structure of the use cases more efficient by collapsing repeated operations into smaller use cases that can be shared among many other use cases. The goal is to manage change and eliminate redundancy. By moving the common parts of many use cases into a single included use case, when the requirements of that common part change, only the single included use case must change.

We use the «extend» relationship when we know that there are many alternatives or options within a use case. We separate the invariant part of the use case from the variable parts. The invariant part becomes the use case that is extended, and the variable parts become the extending use cases. The goal, once again, is change management. In the current example, if new payment options are added, new extending use cases will have to be created, but no existing use cases need to be modified.

Notation for the «include» Relationship. The diagram for Use Case #2 shows that the "Enroll in a Course" use case is bound to the "View Menu of Courses" use case by a dashed line that terminates in an open arrowhead. The arrowhead points at the included use case and has the stereotype «include».

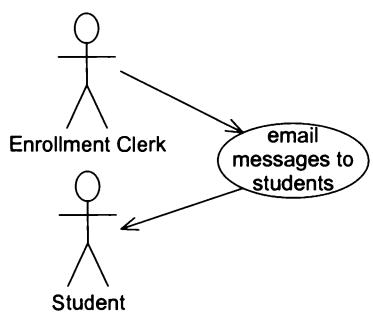


Use Case #2.1: Pay by Purchase Order. The Enroller is prompted for the PO#, the name of the company, and the name and phone number of someone in the accounts-payable department.

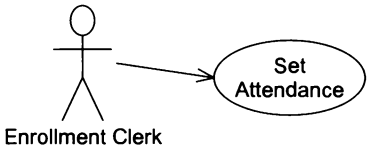
Use Case #2.2: Pay by Check. The Enroller is prompted for the check number.

Use Case #2.3: Pay by Card. The Enroller is prompted for the credit card number, expiration date, and the name as it appears on the card.

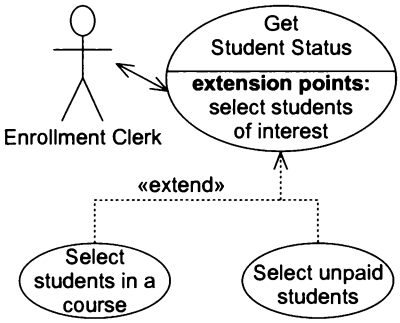
Notation for Use-Case Extension. The extending use cases shown above are connected to the extended use case through «extend» relationships. This relationship is drawn as a line that connects the two use cases. Once again, dashed, the line has an open arrowhead. The arrowhead points at the extended use case and has the stereotype «extend».



Use Case #3: Email Messages to Students. The Enrollment Clerk selects a course and enters the text of a message. The system sends the message to the e-mail addresses of all the students currently enrolled in that class.



Use Case #4: Set Attendance. The Enrollment Clerk selects a class and a student that is currently enrolled in that class. The system presents the student and shows whether that student was in attendance and whether or not that student's payment has been received. The Enrollment Clerk may then change the attendance or payment status.



Use Case #5: Get Student Status. The Enrollment Clerk selects the students of interest.

Extension Point: 'select students of interest'
The system presents the attendance and payment status of the selected students in a single report.

Extensions There are two use cases that extend Get Student Status.

Use Case #5.1: Select Students in a Course. The system shows a list of all the courses. The Enrollment Clerk selects a course. The system selects all students in that course.

Use Case #5.2: Select Unpaid Students. The Enrollment Clerk indicates that the system should select all unpaid students. The system selects all students who have been marked in attendance and whose payment status indicates that payment has not been received.

Use-Case Reprise. The use cases we have created here describe how the users expect the system to behave. Notice that they do not talk about the details of the user interface. They don't mention icons, menu items, buttons, or scrolling lists. Indeed, even the original specification said more about the user interface than the use cases do. This is intentional. We want the use cases to be lightweight and easy to maintain. As written, these use cases are valid for an extremely large set of possible implementations.

System-Boundary Diagram. The entire set of use cases can be shown in summary by the system-boundary-diagram in Figure A-1. This diagram shows all the use cases in the system surrounded by a rectangle that represents the system boundary. The actors are set outside the system and are connected to the use cases with associations that show the direction that data flows.

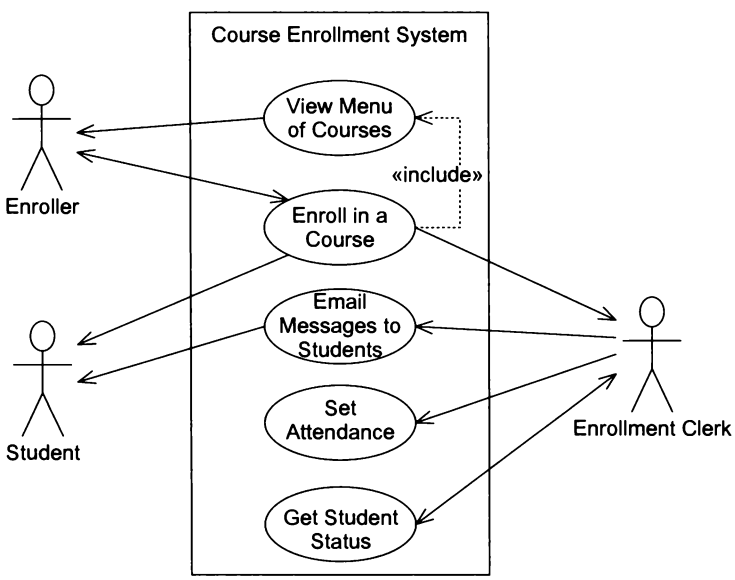


Figure A-1 System-Boundary Diagram

What Do We Use these Diagrams for? Use-case diagrams, including system-boundary-diagrams, are not software-structure diagrams. They do not give us any information at all about the partitioning of the software elements of the system to be created. These diagrams are used for human communication, primarily between the analysts and the stakeholders. They help to organize the functionality of the system in terms of the different kinds of system users.

Moreover, these diagrams can be quite helpful when presenting the system description to the various kinds of users. Each different kind of user is interested chiefly in his own use cases. The connection of the actors to the use cases serves to focus each different kind of user on the use cases that he or she will be using. In very large systems, you may wish to organize the system boundary diagrams according to actor type so that all different user types can look at the subset of use cases that concerns them.

Finally, remember Martin's first law of documentation: *Produce no document unless its need is immediate and significant.*

These diagrams can be useful, but they are often unnecessary. You should not think of them as required or essential. If you have a need for them, then draw them. Otherwise, wait until you have a need.

The Domain Model

The domain model is a set of diagrams that helps to define the terms that appear in the use cases. These diagrams show the key objects within the problem and their interrelationships. Much pain and suffering have resulted from the tendency to assume that this model is also a model of the software to be built. It is important for both the analyst and the designer to realize that a domain model is a descriptive tool that is used to help humans record their decisions and communicate with each other. The objects in the domain model do not necessarily correspond to the object-oriented design of the software, nor would such a correspondence be a strong benefit.[5]

In Booch 94 and OMT, domain-model diagrams were indistinguishable from diagrams that represented software structure and design. In the worst cases, the domain model diagrams were taken to be high-level design documents and were therefore used to set the high-level structure of the software itself.

To help avoid this kind of mistake, we can take advantage of the features of UML and use a special kind of entity called a «type» in domain models. A «type» represents a role that an object can play. A «type» can have operations and attributes as well as associations with other «type» entities. However, a «type» does not represent a class or object in the design sense. It does not represent an element of software, and it does not map directly to code. It represents a conceptual entity used in the description of the problem.

The Course Catalog. The first domain abstraction we will consider is the course catalog. This abstraction represents the list of all the courses that are offered. We show this in the domain model (see Figure A-2) for the `Course Catalog` entity by depicting two entities that represent abstractions in the domain: the `CourseCatalog` entity and the `Course` entity. The `CourseCatalog` entity offers many `Course` entities.

Domain-Model Notation. The notation used in Figure A-2 depicts the two domain abstractions as UML classes with the «type» stereotype. (See sidebar, "Overview of UML Class Notation and Semantics.") This indicates that the classes are conceptual elements of the problem domain and not directly related to software

5. [JACOBSON], p. 133, "We do not believe that the best (most stable) systems are built by *only* using objects that correspond to real-life entities..."

   And also on p. 167, "In [other] methods, this [domain] model will also form a base for the actual implementation; that is, the objects are directly mapped onto classes during implementation. However, this is not the case in OOSE, [...]. Our experience with such an approach tells us differently. Instead we develop an analysis model that is more robust and maintainable in the face of future changes rather than using a problem domain model to serve as the base for design and implementation.

   [BOOCH96], p. 108, "...there is a tendency in immature projects to consider the domain model resulting from analysis as ready to code [...], thus skipping any further design. Healthy projects recognize that there is still quite some work to do, involving issues such as concurrency, serialization, safety, distributions, and so on, and that the design model may end up looking quite different in a number of subtle ways.

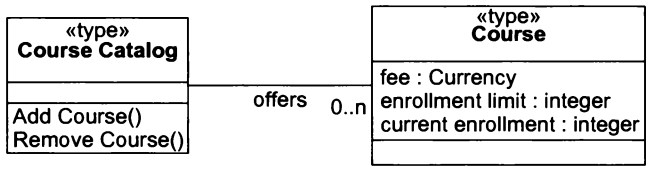


Figure A-2 Domain model for the `Course Catalog` entity

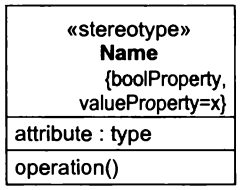
classes. Notice that the `CourseCatalog` entity has two operations: `AddCourse` and `RemoveCourse`. In a «type», operations correspond to *responsibilities*. Thus, the `CourseCatalog` has the responsibility to be able to add and remove courses. Again, these are concepts, not the specifications of member functions within real classes. We use them to communicate with users, not to specify a software structure.

By the same token, the attributes shown in the `Course` entity are concepts. They indicate that the `Course` entity should be responsible to remember its fee, enrollment limit, and current enrollment.

Overview of UML Class Notation and Semantics

In UML, a class is drawn as a rectangle with three compartments. The first compartment specifies the name of the class. The second specifies its attributes and the third its operations.

Within the name compartment, the name can be modified by a stereotype and by properties. The stereotype appears above the name and is enclosed within guillemets (French quotation marks «»). Properties appear below and to the right of the name and are enclosed within braces. (See the following diagram.)



Stereotypes are names that refer to the "kind" of UML class being represented. In UML, a class is simply a named entity that can have attributes and operations. The default stereotype is «implementation class» in which case the UML class corresponds directly to the software notion of a class in a language like C++, Java, Smalltalk, or Eifel. The attributes correspond to member variables, and the operations correspond to member functions.

However, with the stereotype of «type», a UML class does not correspond to a software entity at all. Rather it corresponds to a conceptual entity that exists in the problem domain. The attributes represent the information that logically belongs to that conceptual entity, and the operations represent the responsibilities of that conceptual entity.

There are several other predefined stereotypes that we will be discussing later in this chapter. You are free to create your own stereotypes, too. However, a stereotype is more than just a comment. It specifies the way that all elements of a UML class should be interpreted. So if you make a new stereotype, define it well.

(continues on next page)

Properties are primarily structured comments. Properties are specified in a comma-separated list between braces. Each property is a name = value pair separated by an equal sign (=). If the equal sign is omitted, then the property is assumed boolean and given the value of "true." Otherwise the type of the value is a string.

There are several predefined properties that we will be discussing later in this chapter. However, you are free to add your own at any time. For example, you might create properties like this: {author=Robert C. Martin, date=12/6/97, SPR=4033}

Concept vs. Implementation and the Use of Cloud Icons. I have been placing a lot of stress upon the difference between a class at the conceptual level (i.e., a «type») and a class at the design or implementation level. I feel that this is appropriate because of the danger that conceptual diagrams might be mistaken as specifications for the structure and architecture of the software. Remember, conceptual diagrams are to help communications with the *stakeholders* and are therefore devoid of all the technical issues of software structure.

The use of stereotypes to distinguish between these kinds of diagrams could be overlooked. UML classes for domain models look very much like UML classes for design and implementation. Fortunately, UML allows us to substitute different icons for different stereotypes. Therefore, to further enhance the difference between these kinds of diagrams, we will use a cloud icon to represent «type» classes from now on. This changes the domain model for the `Course Catalog` entity in Figure A-2 to look like Figure A-3.

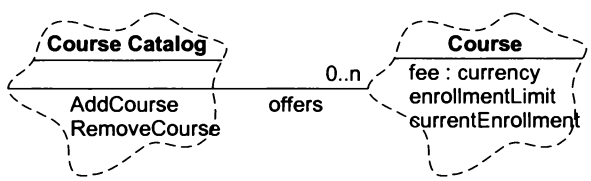


Figure A-3 Domain Model for Course Catalog using Cloud Icons

Completing the Domain Model. So far, the domain model shows a course catalog that contains all the offered courses. But there is a problem here. What do we mean by a course? The same course can be offered at many different times and locations, and it can be taught by many different instructors. Clearly we need two different entities. Let's call the first a `Course`. It represents the course itself, but not the dates, locations, or instructors. Let's call the second a `Session`. It represents the date, location, or instructor for a specific course. (See Figure A-4.)

Notation. The lines that connect the entities are called associations. All the associations in Figure A-4 are named, although this is not a rule. Notice that the names are verbs or verb phrases. The little black triangle next to the name points at the predicate of the sentence formed by the two entities and the association. Thus, "Course Catalog offers many Courses," "Session Schedule schedules many Sessions," and "Many students are enrolled in a Session."

I used the word "many" in the previous sentences wherever the "`0..*`" icon was present on the corresponding relationship. This icon is one of several different *multiplicity* icons that can be placed on the end of an association. They indicate the number of entities that participate in the association. The default multiplicity is "1." (See the sidebar entitled: "Multiplicities.")

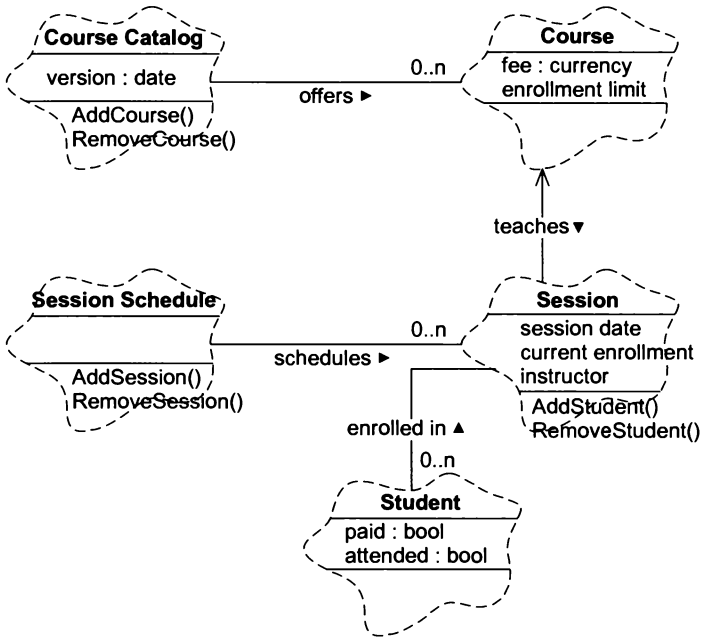


Figure A-4 Courses and Sessions

Multiplicities

There are several multiplicity icons that can be used to adorn associations. They include the following:

- `0..*` zero to many
- `*` zero to many
- `1..*` one to many
- `0..1` zero or one
- `6` exactly six
- `3..6` between three and six
- `3,5` three or five

Any nonnegative integers can be used between dots or separated by commas.

Associations are assumed to be bidirectional unless there is an arrowhead present. A bidirectional association allows the two entities to know about each other. For example, it is clear that the `CourseCatalog` entity ought to know about its `Courses`, and it seems reasonable that each `Course` entity could know about the `CourseCatalog` it is listed in. The same is true for the `SessionSchedule` and the `Session`.

The presence of an arrowhead restricts the knowledge to the direction indicated. Thus, `Sessions` know about `Courses`, but `Courses` do not know anything about `Sessions`.

Use Case Iterations. This diagram immediately tells us two things. First, the use cases employ the wrong language in a number of places. Where they talk about course catalogs and courses, they should be talking about session schedules and sessions. Secondly, there are quite a few use cases that were left out. The `CourseCatalog` and the `SessionSchedule` need to be maintained. `Courses` need to be added and removed from the `CourseCatalog`, and `Sessions` need to be added and removed from the `SessionSchedule`.

Thus, by creating a domain model, we better understand the problem at hand. That better understanding helps us to improve and augment the use cases. This iteration between the two is natural and necessary.

If we were pursuing this case study to its conclusion, we would show the changes implied above. But in the interest of efficiently presenting the notation, we will skip the iteration of the use cases.

The Architecture

Now we get down to the business of designing software. The architecture represents the software structures that form the skeleton of the application. The classes and relationships of the architecture map very closely to code.

Deciding the Software Platform. Before we can begin, however, we must understand the software platform in which this application is going to run. We have a number of choices.

1. A Web-based CGI application. The enrollment and other forms would be accessed by a Web browser. The data would reside at the Web server, and CGI scripts would be invoked by the Web browser to access and manipulate the data.
2. A database application. We could purchase a relational database and use the forms package and 4GL to write the application.
3. Visual XXX. We could purchase a visual programming language. The human interface could be created using the visual construction tools. These tools would invoke the software functions needed to store, retrieve, and manipulate the data.

There are, of course, other options. We could write the whole thing in C without any library or tool support other than the compiler. But this would be silly. The tools are there, and they work. We should use them.

For the purposes of our example, we will assume a Web-based application. This makes sense since then enrollers can be located anywhere in the world; and the enrollment service can be offered on the Internet.

Web Architecture. We need to decide the overall architecture of the Web applications. How many Web pages will there be, and what CGI programs will they invoke? Figure A-5 shows how we might begin to specify this.

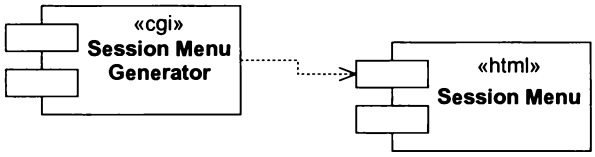


Figure A-5 Session Menu Architecture

Notation. Figure A-5 is a component diagram. The icons depict physical software components. We have used the stereotype to specify the kind of components. The diagram shows that the Session Menu is displayed as an HTML Web page that is generated by a CGI program called the Session Menu Generator. The dashed arrow between the two components is a dependency relationship. Dependency relationships denote which components have knowledge of other components. In this case, the Session Menu Generator program creates the Session Menu Web page, and therefore has knowledge of it. The Session Menu Web page, on the other hand, has no knowledge of the generator itself.

Custom Icons. There are two different kinds of components in Figure A-5. To make them visually distinct, we extend UML with two new icons—one for CGI programs and one for HTML pages. Figure A-6 shows the components involved in Use Case #2: Enroll in a course. Web pages are drawn as pages with a 'W' in them. CGI programs are drawn as Circles with "CGI" in them.

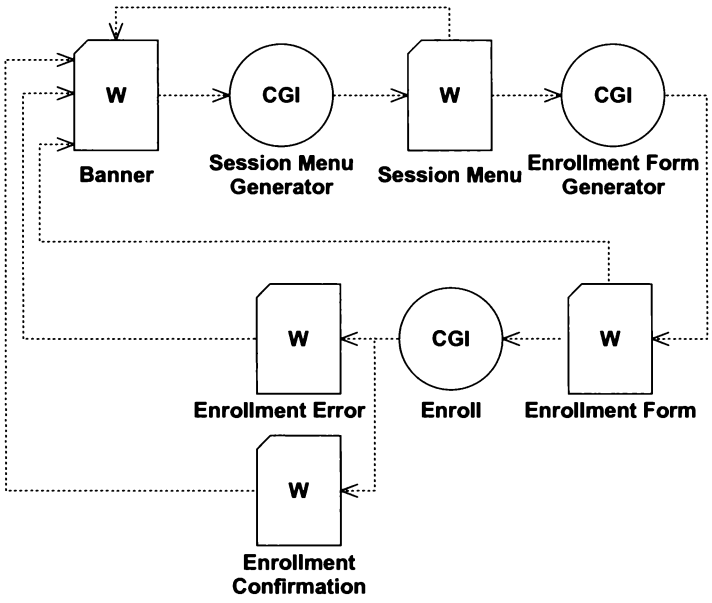


Figure A-6 Enrollment Components

Component Flow. Figure A-6 introduces two new Web pages and one new CGI program. We have decided that the application should begin with a banner page of some kind. Presumably, this page will have links for the various kinds of operations that users can perform. The Session Menu Generator is invoked by the banner page and generates the Session Menu page. Presumably the Session Menu page has links or buttons that allow the user to enroll in a course. The Session Menu invokes the Enrollment Form Generator CGI to create the form necessary for enrolling in the selected course. Once the user fills out this form, the Enroll CGI is invoked. This program validates and records the information on the form. If the form data are invalid, it generates the Enrollment Error page; otherwise it generates the confirmation page and sends the necessary e-mail messages. (Refer back to Use Case #2.)

Enhancing Flexibility. The astute reader will have recognized that there is a rather nasty inflexibility built into this component model. The CGI programs generate most of the Web pages. This means that the HTML text within the Web pages must be contained within the CGI programs. This makes modifying the Web pages a matter of modifying and rebuilding the CGI programs. We would rather the bulk of the generated Web pages were created with a nice HTML editor.

Thus, the CGI programs should read in a template of the Web pages that they are going to generate. The template should be marked with special flags, which will be replaced with the HTML that the CGI programs must generate. This means that the CGI programs share something in common. They all read template HTML files and add their own bits of HTML to them.[6]

Figure A-7 shows the resultant component diagram. Notice that I have added a WT icon. This represents an HTML template. It is a text file in HTML format with the special marks that the CGI programs use as insert points for the HTML that they generate. Notice also the direction of the dependency relationships between the CGI programs and the HTML templates. They might appear backwards to you. But remember they are *dependency* relationships, not data flows. The CGI programs know about (depend on) the HTML templates.

6. This chapter was written long before XSLT came on the scene. Nowadays, we'd likely solve the problem by having the CGI scripts (or servlets) generate XML and then invoke an XSLT script to translate that into HTML. On the other hand, generating HTML with XSLT still doesn't give us the ability to design our Web pages with a nice WYSIWYG editor. Sometimes I think the template scheme outlined in this chapter would be better in many instances.

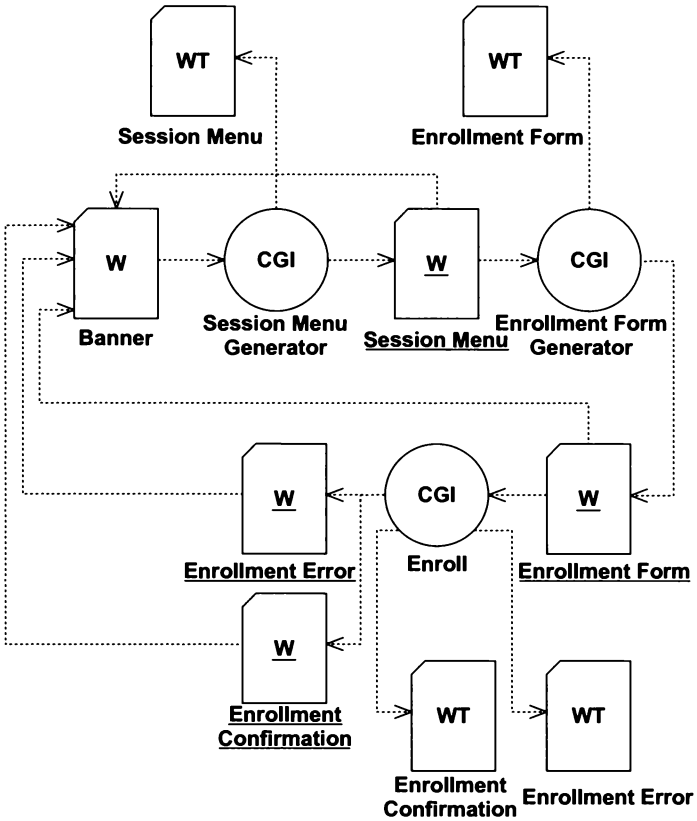


Figure A-7 Adding HTML templates to the Enrollment Component Diagram

The Specification/Instance Dichotomy. In Figure A-7, the names of the generated Web pages are underlined. This is because these pages only exist at run time. They are instances of the HTML templates. In UML, the convention is that we underline instances. An instance is a software element that is generated from a specification (source document) of some kind. We'll have more to say about this later. For now, simply realize that the elements whose names are not underlined represent elements that must be written by hand and that act as specifications. Elements whose names are underlined are the product of some process that generates them from those specifications.

Using the HTML Templates. The HTML templates provide a great deal of flexibility to the architecture of this application. How do they work? How do the CGI programs place their generated output in the proper places within the HTML Templates?

We might consider placing a special HTML tag in the HTML template files. Such a tag would mark the position in the generated HTML file where the CGI program would insert its output. However, these generated Web pages may have several sections, each of which might require its own insertion point into which the CGI can insert HTML. Thus, each HTML template could have more than one insertion tag, and the CGI will have to be able to somehow specify which output goes with which tag.

The tag could look like this: `<insert name>` where "name" is an arbitrary string that identifies the insertion point to the CGI. A tag such as `<insert header>` will allow a CGI to specify the name 'header' and then completely replace the tag with generated output.

Clearly each tag is replaced with a string of characters, so each tag represents a kind of named stream of characters. We can imagine the kind of C++ code that might exist in the CGI as follows:

```
HTMLTemplate myPage("mypage.htmp");
myPage.insert("header",
              "<h1> this is a header </h1>\n");
cout << myPage.Generate();
```

This would send HTML to `cout` that was generated from the template `mypage.html` in which the tag `<insert header>` was replaced with the string `"<h1> this is a header </h1>\n"`.

Figure A-8 shows how we might design the `HTMLTemplate` class. This class holds the name of the template file as an attribute. It also has methods that allow replacement strings to be inserted for specifically named insertion points. Instances of `HTMLTemplate` will contain a `map` that relates the insertion point name to the replacement string.

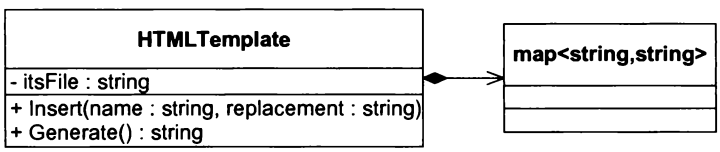


Figure A-8 `HTMLTemplate` design

Notation. This is our first true class diagram. It shows us two classes related with a *composition* relationship. The icon used for the `HTMLTemplate` class and the `map<string,string>` class is not new to us. It was explained in the sidebar named "Overview of UML Class Notation and Semantics" on page 473. The syntax of the attributes and operations is described below in the sidebar entitled, "Attributes and Operations."

Attributes and Operations

Attributes and operations can be adorned with the following encapsulation specifiers:

+	Public
-	Private
#	Protected

The type of an attribute can be specified as an identifier that follows the attribute and is separated from it with a colon (e.g., `count : int`).

Similarly, the types of the arguments of a function are specified using the same colon notation (e.g., `SetName (name : string)`).

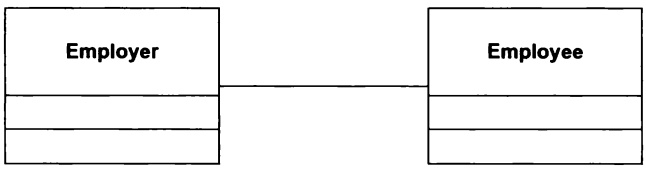
Finally, the return type of an operation can be specified with an identifier that follows the name and argument list of the operation and is separated from it by a colon (e.g., `Distance(from : Point) : float`).

The arrowhead on the association that connects the two classes in Figure A-8 indicates that `HTMLTemplate` knows about `map<string,string>`, but that the map does not know about the `HTMLTemplate`. The black diamond on the end of the association nearest the `HTMLTemplate` class identifies this as a special case of association called composition. (See the sidebar entitled, "Association, Aggregation, and Composition.") It indicates that `HTMLTemplate` is responsible for the lifetime of the `map` class.

Association, Aggregation, and Composition

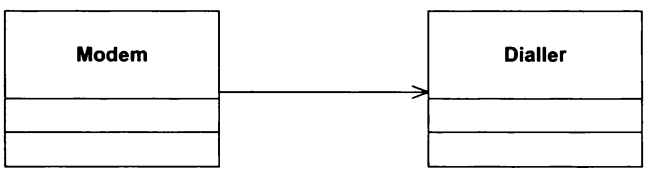
An association is a relationship between two classes that allows instances created from those classes to send messages to each other. (i.e., links may exist between objects whose classes are associated). It is denoted by a line that connects the two classes. Associations are most often implemented as instance variables in one class that point or refer to the other.

Association



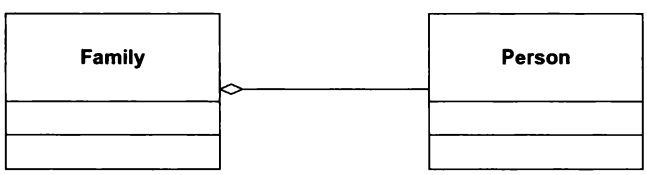
The navigability of an association can be restricted by adding arrowheads to the associations. When an arrowhead is present, the association can only be navigated in the direction of the arrow. This means that the class to which the arrow points does not know about its associate.

Navigable Association



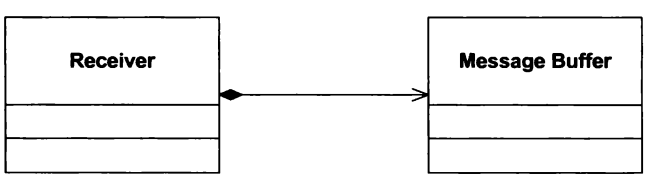
Aggregation is a special form of association. It is denoted with a white diamond on the aggregate class. Aggregation implies a "whole/part" relationship. The class adjacent to the white diamond is the "whole," and the other class is its "part." The "whole/part" relationship is purely connotative; there is no semantic difference from association[a].

Aggregation



Composition is a special form of aggregation. It is denoted with a black diamond. It implies that the "whole" is responsible for the lifetime of its "part." This responsibility does not imply either creation or deletion responsibility. Rather, it implies that the "whole" must see to it that the "part" is somehow deleted. That can be accomplished by directly deleting the "part," or by passing the "part" to another entity that assumes responsibility for it.

Composition



a. With one exception. Reflexive or cyclic aggregation relationships between *objects* is not allowed. That is, instances cannot participate in a cycle of aggregations. If this rule were not in place, all instances in the cycle would be part of themselves. That is, a part could contain its whole.

Note that this rule does not prevent *classes* from participating in a cycle of aggregations; it simply restricts their instances.

The Database Interface Layer. Each CGI programs must also have access to the data that represent the courses, classes, students, etc. We shall call this the training database. The form of this database is, as yet, undecided. It could be held in a relational database or in a set of flat files. We do not want the architecture of our application to depend on the form in which the data are stored. We would like the bulk of each application to remain unchanged when the form of the database changes. Thus, we will shield the application from the database by interposing a database interface layer (DIL).

In order to be effective, a DIL must have the special dependency characteristics shown in Figure A-9. The DIL depends on the application, and the DIL depends on the database. Neither the application nor the database has any knowledge of each other. This allows us to change the database without necessarily changing the application. It also allows us to change the application without changing the database. We can completely replace the database format or engine without affecting the application.

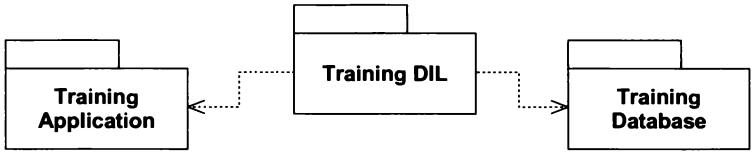
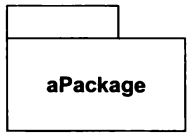


Figure A-9 Database Interface Layer Dependency Characteristics

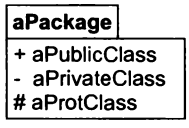
Notation. Figure A-9 shows a special kind of class diagram called a "package diagram." The icons denote packages. Their shape is reminiscent of a file folder. Like a file folder, a package is a container. (See the "Packages and Subsystems" sidebar.) The packages in Figure A-9 contain software components such as classes, HTML files, CGI main program files, etc. The dashed arrow that connects the packages represents a *dependency* relationship. The arrowhead points at the target of the dependency. A dependency between packages implies that the dependent package cannot be used without the package on which it depends.

Packages and Subsystems

Packages are drawn as a large rectangle with a smaller rectangular "tab" on the upper left of the large rectangle. Normally, the name of the package is placed in the large rectangle.



Packages can also be drawn with the package name in the "tab" and the contents of the package in the large rectangle. The contents may be classes, files, or other packages. Each can be prefixed with the (`-`, `+`, `#`) encapsulation icons to denote that they are private, public, or protected within the package.

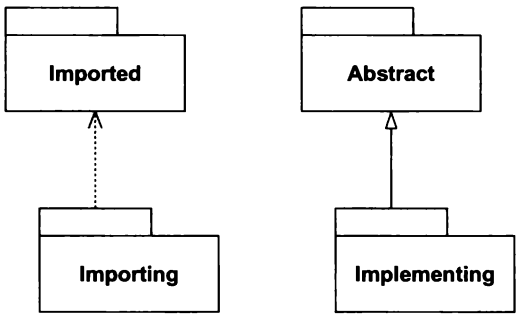


Packages may be connected by one of two different relationships. The dashed dependency arrow is called an *import dependency*. It is a dependency relationship with a stereotype of «import». This

(continues on next page)

stereotype is the default when a dependency is used with a package. The base of the arrow is attached to the importing package, and the arrowhead touches the imported package. An import dependency implies that the importing package has visibility to any of the public elements of the imported package. This means that the elements of the importing package can use any public element of the imported package.

Packages can also be connected with Generalization relationships. The open triangular arrowhead touches the general or abstract package, and the other end of the relationship touches the implementing package. The implementing package has visibility to any public or protected element of the abstract package.



There are several defined stereotypes of package. The default is «package», which denotes a container without any special constraints. It can hold anything that can be modeled in UML. Typically, it is used to denote a physical, *releaseable*, unit. Such a package would be tracked in a configuration management and version control system. It might be represented by subdirectories in a file system or by the module system of a language (e.g., Java packages or JAR files). Packages represent a partitioning of the system that enhances its developability and releasability.

The «subsystem» stereotype for packages denotes a logical element that, in addition to containing model elements, also specifies their behavior. A subsystem can be given operations. Those operations must be supported by use cases or collaborations within the package. Subsystems represent a *behavioral* partitioning of a system or application.



These two kinds of packages are orthogonal to each other. The partitioning that enhances developability and releasability is almost never similar to a partitioning that is based upon behavior. The former is often used by software engineers as the unit of configuration management and version control. The latter is used more often by analysts in order to describe the system in an intuitive fashion and to perform impact analyses when features change or are added.

Database Interface. The classes within the Training Application package need some way to access the database. This will be accomplished through a set of *interfaces* inside the Training Application package. (See Figure A-10.) These interfaces represent the types in the domain model of Figure A-4 on page 475. The interfaces are implemented by classes in the DIL package. They will be used by the other classes within the Training Application

package to access the data in the database. Notice that the direction of the dependencies in Figure A-10 corresponds to the direction of the import relationships between the packages in Figure A-9.

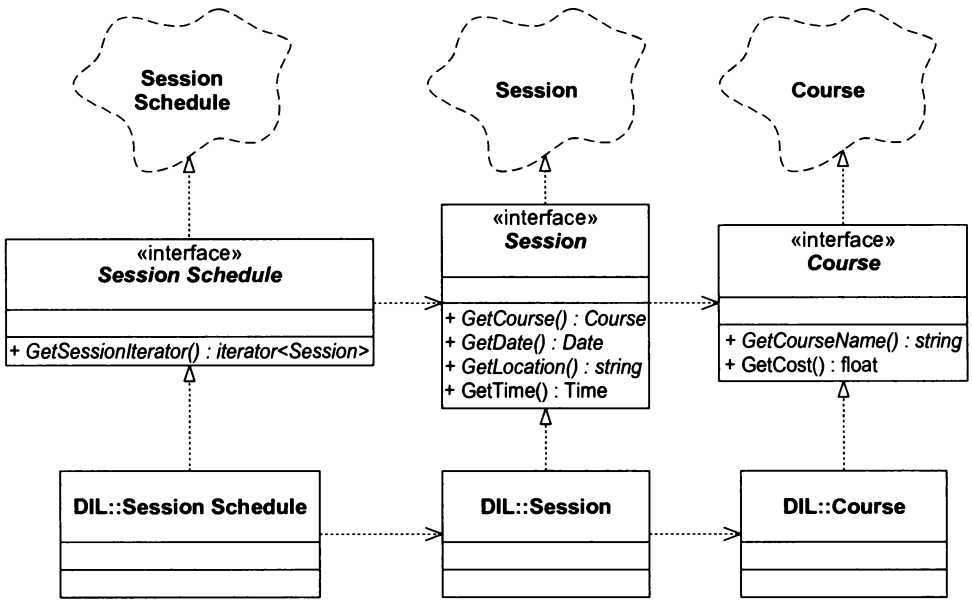


Figure A-10 Database interface classes in the Training Application package

Notation. When the name of a class is written in italics, it indicates that the class is abstract.[7] Since interfaces are completely abstract classes, it is appropriate to italicize their names. The operations are also shown in italics to denote that they are abstract. The classes in the DIL are bound to the interfaces through *realizes* relationships, which are drawn as dashed lines with open triangular arrowheads pointing at the interfaces. In Java, these would represent "`implements`" relationships. In C++, they would represent inheritance.

Interfaces are physical structures. In languages like C++ and Java, they have source code counterparts. Types, on the other hand, are not physical and do not represent something that has a source-code equivalent. In Figure A-10, we have drawn realizes relationships from the interfaces to the types that they represent. This does not represent a physical relationship, nor is there any corresponding source code. In this case, the realizes relationship is showing the correspondence between the physical design entities and the domain model. This correspondence is seldom as clear-cut as depicted here.

The *imports* relationships between the packages in Figure A-9 are shown in Figure A-10 through the use of the double colon. The `DIL::Session` class is a class named `Session` that exists in the `DIL` package and is visible to (has been imported into) the `TrainingApplication` package. The fact that there are two classes named `Session` is acceptable since they are in different packages.

The Session Menu Generator. Referring back to Figure A-7, we see that the first CGI program is the `SessionMenuGenerator`. This program corresponds to the very first use case back on page 469. What does the design of this CGI program look like?

Clearly it must build an HTML representation of the session schedule. Thus, we will need an `HTMLTemplate` that merges the boiler plate of the session menu with the actual data from the session schedule. Also, the program will use the `SessionSchedule` interface to access the `Session` and `Course` instances in the database in order to get their names, times, locations, and fees. Figure A-11 shows a sequence diagram that describes the process.

7. An abstract class has at least one pure (or abstract) method.

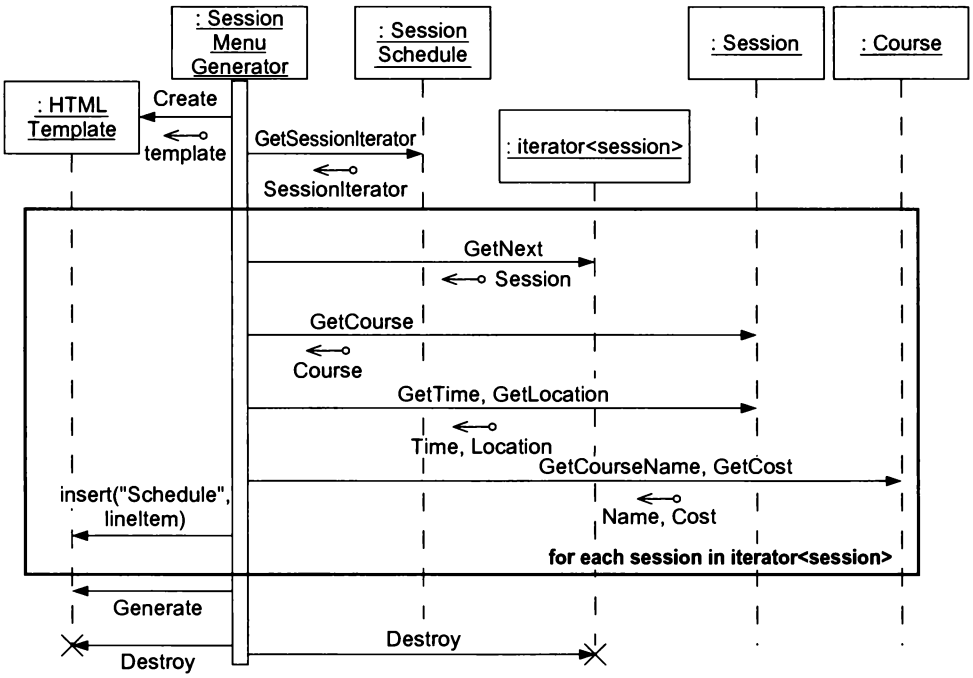


Figure A-11 `SessionMenuGenerator` Sequence Diagram

The `SessionMenuGenerator` object is created by `main` and controls the entire application. It creates an `HTMLTemplate` object passing it the name of the template file. It then fetches an `iterator<Session>`[8] object from the `SessionSchedule` interface. It loops through every `Session` in the `iterator<Session>` asking each `Session` for its `Course`. It fetches the time and location of the course from the `Session` object, and the name and cost of the course from the `Course` object. The last action in the loop is to create a line item from all that information and `Insert` it into the `HTMLTemplate` object at the `Schedule` insertion point. When the loop is completed, the `SessionMenuGenerator` invokes the `Generate` method of the `HTMLTemplate` and then destroys it.

Notation. The names within the rectangles on the sequence diagram in Figure A-11 are underlined. This indicates that they represent objects rather than classes. Object names are composed of two elements separated by a colon. Prior to the colon is the *simple name* of the object. Following the colon is the name of a class or interface that the object implements. In Figure A-11, the simple names are all omitted, so all the names begin with a colon.

The dashed lines hanging down from the objects are called *lifelines*, and they represent the lifetime of the objects. All the objects in Figure A-11, except `HTMLTemplate` and `iterator<Session>`, show lifelines that start at the top and end at the bottom. By convention, this means that those objects were in existence before the scenario began and remain in existence when the scenario ends. `HTMLTemplate`, on the other hand, is explicitly created and destroyed by `SessionMenuGenerator`. This is evident by the arrow that terminates on `HTMLTemplate`, thus creating it, and the "X" that terminates its lifeline at the bottom. `SessionMenuGenerator` also destroys `Iterator<Session>`; however, it is not clear what object creates it. The creator is probably a derivative of `SessionSchedule`. Thus, while Figure A-11 does not explicitly show the creation of the `Iterator<Session>` object, the position of the start of the object's lifeline implies that it is created at about the time that the `GetSessionIterator` message is sent to the `SessionSchedule` object.

The arrows that connect the lifelines are messages. Time proceeds from top to bottom, so this diagram shows the sequence of messages that are passed between the objects. The messages are named by labels close to the

8. This chapter was written before STL was commonplace. At the time I was using my own container library, which had templated iterators.

arrows. The short arrows with circles on the end are called *data tokens*. They represent data elements that are passed in the context of the message. If they point in the direction of the message, they are parameters of the message. If they point against the direction of the message, they are values returned by the message.

The long, skinny rectangle on the `SessionMenuGenerator` lifeline is called an *activation*. An activation represents the duration of the execution of a method or function. In this case, the message that started the method is not shown. The other lifelines in Figure A-11 do not have activations because the methods are all very short and do not emit other messages.

The bold rectangular box that surrounds some messages in Figure A-11 defines a loop. The completion criterion of the loop is mentioned at the bottom of the box. In this case, the enclosed messages will repeat until all the `Session` objects within `iterator<Session>` have been examined.

Note that two of the message arrows have been overloaded with more than one message. This is just a shorthand to minimize the number of arrows. The messages are sent in the order mentioned, and the return values come back in the same order.

Abstract Classes and Interfaces in Sequence Diagrams

The astute reader will notice that some of the objects in Figure A-11 are instantiated from interfaces. `SessionSchedule`, for example, is one of the database interfaces classes. This may seem to be violating the principle that objects cannot be instantiated from abstract classes or interfaces.

The class name of an object in a sequence diagram does not need to be the name of the actual type of the object. It is sufficient that the object simply conform to the interface of the named class. In a static language like C++, Java, or Eifel, the object should belong either to the class named in the sequence diagram or to a class that derives from the class or interface in the sequence diagram. In dynamic languages like Smalltalk or Objective-C, it is sufficient that the object conform to the interface named in the sequence diagram.[9]

Thus, the `SessionSchedule` object in Figure A-11 refers to an object whose class implements or derives from the `SessionSchedule` interface.

Static Model of the Session Menu Generator. The dynamic model shown in Figure A-11 implies the static model shown in Figure A-12. Note that the relationships are all either dependencies or stereotyped associations. This is because none of the classes shown in the figure hold instance variables that refer to the others. All the relationships are transient in that they do not outlast the execution of the activation rectangle on the `SessionMenuGenerator` lifeline of Figure A-11.

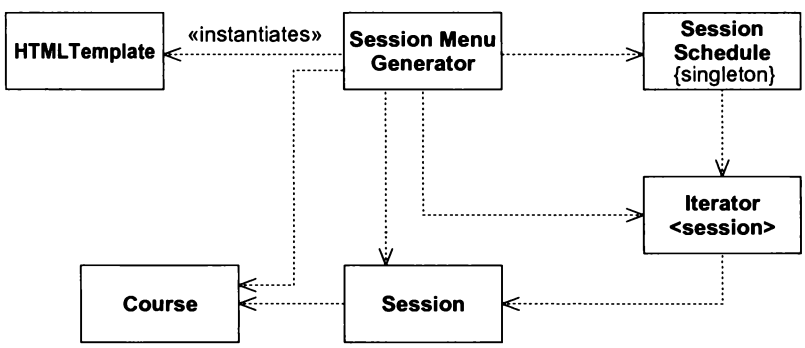


Figure A-12 Static Model of the Session Menu Generator application

9. If you don't understand this, don't worry about it. In dynamic languages like Smalltalk and Objective-C, you can send any message you like to any object you like. The compiler does not check to see whether the object can accept that message. If, at run time, a message is sent to an object that does not recognize it, a run-time error will occur. Thus, it is possible for two completely different and unrelated objects to accept the same messages. Such objects are said to conform to the same interface.